Servicios Web y REST

Desarrollo de Software - ISI - FRRe - UTN
Dr. Jorge E. Villaverde

HTTP · recursos · contratos · seguridad

Objetivos y recorrido

- Comprender la comunicación cliente-servidor mediante HTTP.
- Diseñar recursos y endpoints con criterios REST.
- Interpretar requests, responses y errores en JSON.
- Probar una API y justificar decisiones de diseño.

Evolución de los servicios web

- Integración entre aplicaciones: sistemas heterogéneos intercambian datos y operaciones.
- SOAP: mensajes XML con contratos y protocolos estandarizados.
- REST: estilo arquitectónico que aprovecha la interfaz uniforme de la Web.
- Hoy: HTTP y JSON dominan muchas APIs; SOAP y otros protocolos siguen vigentes según el contexto.

1990s–2000s
Web y XML

SOAP / WSDL
Contratos formales

REST / HTTP
Recursos y representaciones

APIs actuales
Servicios distribuidos

Arquitectura cliente-servidor

Un intercambio atraviesa capas con responsabilidades distintas.



La interfaz desacopla al cliente de la implementación interna.

HTTP: solicitud y respuesta

- El cliente envía un método, un destino y metadatos.
- El servidor interpreta la solicitud y devuelve un estado.
- La respuesta puede incluir headers y una representación.
- Proxies, gateways y cachés pueden intervenir en el recorrido.

```
REQUEST
GET /api/v1/products/42 HTTP/1.1
Host: api.ejemplo.edu
Accept: application/json

RESPONSE
HTTP/1.1 200 OK
Content-Type: application/json

{"id":42,"name":"Teclado"}
```

URI, ruta y parámetros

- La ruta identifica el recurso o colección.
- La query string refina una consulta: filtros, página u orden.
- El path identifica un elemento concreto.
- Los valores deben codificarse y validarse.

```
GET /api/v1/products/42

GET /api/v1/products?category=audio&page=0&size=20

Path: /products/42
Query: category, page, size
```

Headers y body

- Accept expresa qué formato puede procesar el cliente.
- Content-Type describe el formato enviado.
- Authorization transporta credenciales o tokens.
- El body suele contener JSON en POST, PUT o PATCH.
- GET normalmente no lleva body en una API.

```
POST /api/v1/products HTTP/1.1
Accept: application/json
Content-Type: application/json
Authorization: Bearer <token>

{"name":"Mouse","price":25000}
```

Métodos HTTP y semántica

- GET consulta una representación.
- POST procesa una solicitud o crea un recurso.
- PUT reemplaza la representación objetivo.
- PATCH aplica cambios parciales.
- DELETE solicita eliminar el recurso.

GET lectura
POST creación / acción
PUT reemplazo
PATCH modificación parcial
DELETE eliminación

La semántica del método importa para clientes, cachés y reintentos.

Códigos de estado

- 2xx: la solicitud se procesó correctamente.
- 3xx: el cliente debe considerar una redirección.
- 4xx: problema en la solicitud o sus permisos.
- 5xx: el servidor no pudo completar una solicitud válida.

```
200 OK          consulta exitosa
201 Created     recurso creado
204 No Content  sin body de respuesta
400 Bad Request entrada inválida
401 Unauthorized falta autenticación
403 Forbidden   permiso insuficiente
404 Not Found   recurso inexistente
409 Conflict    conflicto de estado
500 Server Error fallo inesperado
```

SOAP y REST

SOAP

- Protocolo con mensajes XML
- WSDL puede describir contratos
- Extensiones estandarizadas
- Adecuado cuando se requiere ese ecosistema

REST

- Estilo arquitectónico
- Recursos e interfaz uniforme
- Representaciones como JSON
- Ligero para muchas APIs web

La elección depende del contrato, interoperabilidad y necesidades del sistema.

Restricciones del estilo REST

- Cliente-servidor: separa interfaz y almacenamiento.
- Sin estado de sesión en el servidor entre requests.
- Caché: las respuestas indican si se pueden reutilizar.
- Interfaz uniforme y recursos identificados por URI.
- Sistema por capas; código bajo demanda como restricción opcional.

REST describe restricciones arquitectónicas.

Una API que usa HTTP y JSON puede seguir convenciones REST sin cumplir plenamente el estilo.

Recursos y URIs

- Nombrar entidades del dominio: products, orders, customers.
- Usar sustantivos en plural para colecciones.
- Representar relaciones con subrecursos cuando aporten claridad.
- Mantener rutas estables y predecibles.

Colección
/api/v1/products

Elemento
/api/v1/products/42

Relación
/api/v1/orders/9001/items

CRUD mapeado a HTTP

Acción del cliente	Método + recurso	Resultado esperado
Leer colección	`GET /products`	200 + lista
Leer elemento	`GET /products/42`	200 + producto
Crear	`POST /products`	201 + Location
Reemplazar	`PUT /products/42`	200 o 204
Eliminar	`DELETE /products/42`	204

JSON como representación

- Objetos agrupan propiedades; arrays representan colecciones.
- Tipos comunes: string, number, boolean, null, object y array.
- JSON no define el contrato por sí solo: documentar campos, rangos y reglas.
- Usar nombres y convenciones consistentes.

```
{
  "id": 42,
  "name": "Teclado mecánico",
  "price": 78500.50,
  "active": true,
  "category": {"id": 3, "name": "Periféricos"}
}
```

API de productos: consulta y creación

- GET devuelve una colección paginada o un recurso.
- POST crea el producto y responde 201 Created.
- Location identifica el URI nuevo.
- Validar tipo, formato, rango y reglas del dominio.

```
POST /api/v1/products
Content-Type: application/json

{"name":"Teclado","price":78500.5,"stock":12}

201 Created
Location: /api/v1/products/42
Content-Type: application/json

{"id":42,"name":"Teclado","price":78500.5,"stock":12}
```

API de órdenes: flujo de recursos

- Una orden tiene identidad y estado propios.
- Los ítems conectan productos con cantidades y precios de compra.
- POST crea la orden; GET consulta su estado.
- El servidor calcula importes y aplica reglas, no confía en totales del cliente.

```
POST /api/v1/orders

{"customerId":17,"items":[
 {"productId":42,"quantity":2},
 {"productId":7,"quantity":1}
]}

201 Created
Location: /api/v1/orders/9001

GET /api/v1/orders/9001
→ status: "PENDING"
```

Diseño de endpoints

- Colecciones y elementos usan el mismo patrón.
- El método expresa la operación.
- Evitar verbos en rutas cuando el cambio es sobre un recurso.
- Las acciones de dominio pueden modelarse como subrecursos si hace falta.

Preferible
POST /api/v1/orders
DELETE /api/v1/products/42

Evitar como convención general
/getProducts
/deleteProduct?id=42

Acción específica
POST /api/v1/orders/9001/cancellation

Respuestas y errores consistentes

- Mantener un esquema común para errores de validación y dominio.
- Incluir un código estable, un mensaje y detalles accionables.
- No exponer stack traces, secretos ni datos internos.
- Usar el status HTTP apropiado junto con el body.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type":"/problems/validation",
  "title":"Datos inválidos",
  "status":422,
  "detail":"Revisá los campos enviados",
  "errors":[{"field":"price","code":"must_be_positive"}]
}
```

Idempotencia y reintentos

- Una operación idempotente produce el mismo efecto al repetirse.
- GET, PUT y DELETE son idempotentes por semántica HTTP.
- POST no es idempotente por defecto.
- Una clave de idempotencia puede proteger creaciones ante reintentos de red.

```
PUT /api/v1/products/42
{"name":"Teclado","price":79000}
```

Repetir el mismo PUT deja el recurso con el mismo estado final.

```
POST /api/v1/orders
Idempotency-Key: 6f3a…
```

El servidor conserva el resultado asociado a esa clave.

Paginación, filtros y orden

- Limitar el tamaño de página para controlar carga.
- Definir si el índice comienza en 0 o en 1.
- Documentar total, cursor y orden por defecto.
- Validar los campos permitidos para ordenar.

```
GET
/api/v1/products?category=audio&minPrice=10000&sort
=price,asc&page=0&size=20

{
  "content":[{"id":42,"name":"Teclado"}],
  "page":0,"size":20,"totalElements":157,
  "totalPages":8
}
```

Versionado de APIs

- Versionar cuando cambia el contrato de forma incompatible.
- La versión en ruta es explícita y simple de operar.
- Cambios aditivos suelen ser compatibles si clientes toleran campos nuevos.
- Publicar política de deprecación y fechas de retiro.

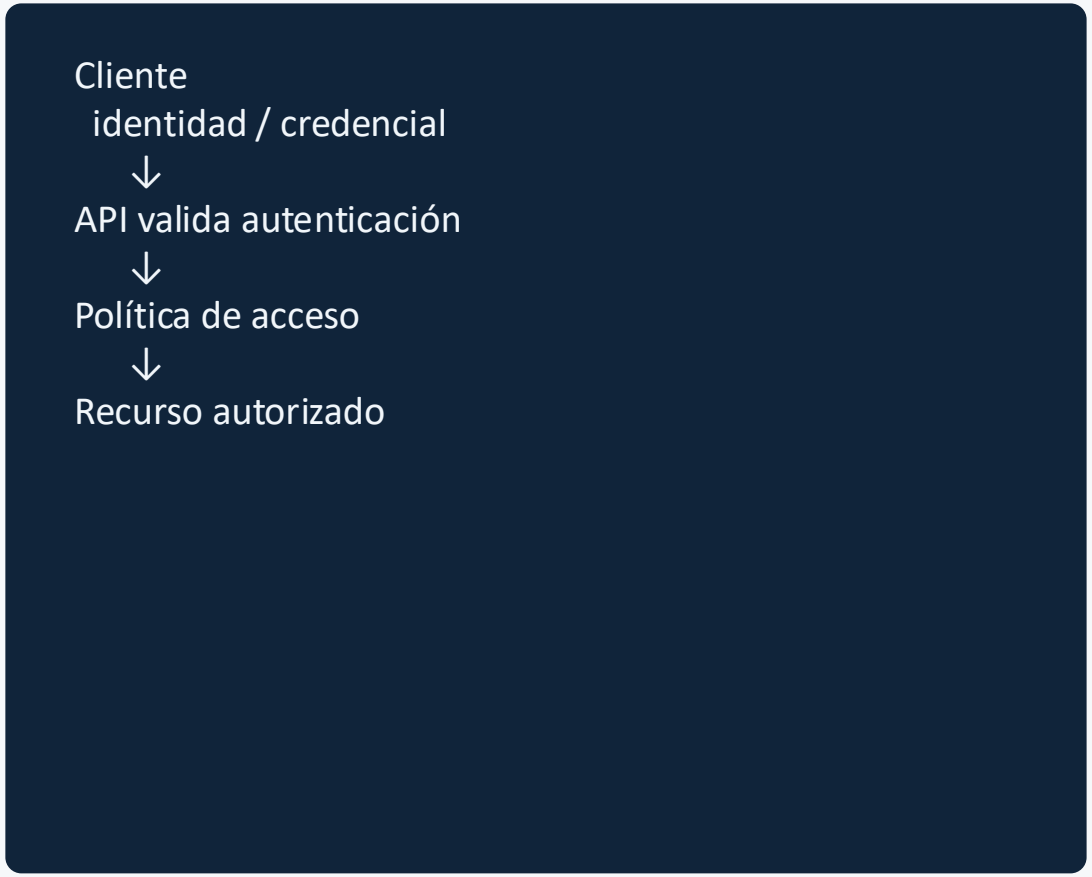
/api/v1/products

Cambios compatibles
Agregar campo opcional

Cambios incompatibles
Renombrar campo requerido
Cambiar significado o formato

Autenticación y autorización

- Autenticación: quién realiza la solicitud.
- Autorización: qué puede hacer esa identidad.
- Aplicar mínimo privilegio en cada recurso y operación.
- Usar HTTPS para proteger credenciales en tránsito.



Basic, API Key, OAuth 2.0 y JWT

- Basic codifica usuario y contraseña en Base64; exige HTTPS.
- API Key identifica o limita una aplicación, no reemplaza autorización de usuario.
- OAuth 2.0 delega autorización mediante access tokens.
- JWT es un formato compacto de claims; firmar no equivale a cifrar.
- Definir expiración, audiencia, scopes y rotación según riesgo.

Basic
Authorization: Basic …

API Key
X-API-Key: …

Bearer token
Authorization: Bearer <access_token>

OAuth 2.0 = marco de autorización
JWT = formato de token

CORS en aplicaciones web

- El navegador restringe requests cross-origin según la política del servidor.
- CORS define headers que habilitan acceso desde otros orígenes.
- Algunos requests activan un preflight OPTIONS.
- CORS no autentica usuarios ni reemplaza controles de acceso.

Origin: https://app.ejemplo.edu

Access-Control-Allow-Origin: https://app.ejemplo.edu
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Headers: Authorization, Content-Type

Con credenciales, configurar origen explícito.

OpenAPI y Swagger

- OpenAPI describe endpoints, parámetros, esquemas, respuestas y seguridad.
- La especificación ayuda a generar documentación y clientes.
- Swagger UI presenta una interfaz interactiva a partir del documento.
- El contrato debe mantenerse alineado con el comportamiento real.

```
openapi: 3.1.0
paths:
  /products:
    get:
      responses:
        "200":
          description: Lista de productos
```

Contrato legible por personas y herramientas

Prueba rápida con curl

- Construir el request con método, URL y headers.
- Inspeccionar status y headers de respuesta.
- Enviar JSON con Content-Type correcto.
- No incluir claves reales en material compartido.

```
curl -i \

"http://localhost:8080/api/v1/products?page=0&size=20"
\
  -H "Accept: application/json"

Crear producto:
curl -i -X POST \
  http://localhost:8080/api/v1/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Mouse","price":25000}'
```

Pruebas con Postman

- Organizar requests por colección y ambiente.
- Usar variables para host, token e identificadores.
- Verificar status, headers y estructura del body.
- Probar casos exitosos, validaciones y permisos insuficientes.

GET /api/v1/products/42

Checks sugeridos
Status = 200
Content-Type contiene application/json
Body.id = 42

Repetir con ID inexistente
Esperado: 404 + error consistente

Buenas prácticas de diseño

- Diseñar desde el dominio y los casos de uso.
- Mantener semántica HTTP y contratos consistentes.
- Validar entradas en el borde y reglas en el dominio.
- Proteger datos, limitar consumo y observar errores.
- Documentar ejemplos reales y automatizar pruebas del contrato.

Revisión de diseño

¿El recurso tiene una URI estable?
¿El método expresa la intención?
¿El status representa el resultado?
¿El error ayuda a corregir el request?
¿El contrato coincide con la implementación?

Actividad práctica · 10 minutos

En parejas · diseñen una API para una biblioteca universitaria.

ENTREGABLE

1. Definir recursos para libros y préstamos.
2. Escribir 3 endpoints con método y URI.
3. Crear un request JSON para iniciar un préstamo.
4. Definir respuestas de éxito y dos errores.
5. Explicar una decisión de autenticación.

10 MIN

7 min diseño
3 min puesta en común

Criterio: coherencia entre recurso, método, estado HTTP y contrato.

Preguntas para discutir

- ¿Cuándo conviene crear /orders/{id}/cancellation en lugar de DELETE?
- ¿Qué dato calcula el servidor al crear una orden y por qué?
- ¿Qué respuesta usarían si el stock no alcanza?
- ¿Cuándo necesita una API versión nueva?

Cierre

Una API clara ofrece recursos reconocibles, semántica HTTP predecible y contratos que los clientes pueden probar.

¿Preguntas?

Referencias técnicas

HTTP Semantics · RFC 9110

https://www.rfc-editor.org/rfc/rfc9110.html

Fielding · REST architectural style

https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

SOAP 1.2 · W3C

https://www.w3.org/TR/soap12-part1/

JSON · RFC 8259

https://www.rfc-editor.org/rfc/rfc8259.html

Basic · RFC 7617

https://www.rfc-editor.org/rfc/rfc7617.html

OAuth 2.0 · RFC 6749

https://www.rfc-editor.org/rfc/rfc6749.html

JWT · RFC 7519

https://www.rfc-editor.org/rfc/rfc7519.html

OAuth 2.0 Security BCP · RFC 9700

https://www.rfc-editor.org/rfc/rfc9700.html

Fetch Standard · CORS

https://fetch.spec.whatwg.org/#http-cors-protocol

OpenAPI Specification

https://spec.openapis.org/oas/latest.html

curl · HTTPS scripting

https://curl.se/docs/httpscripting.html

Postman · Sending requests

https://learning.postman.com/docs/getting-started/first-steps/sending-the-first-request/

Los ejemplos de productos, órdenes y paginación son ilustrativos.